
Curriculum Ordering and Out-of-Distribution Generalization in Proximal Policy Optimization

Keisuke Nishioka

github.com/keisuke58/rl-curriculum-ood

Abstract

Curriculum learning proposes ordering training from easy to hard to accelerate convergence in RL, yet its effect on out-of-distribution (OOD) generalization is largely unstudied. We train PPO agents under five curriculum conditions in MiniGrid and evaluate them zero-shot on five held-out environments. Counter to the standard narrative, progressive (easy→hard) curriculum performs *worst* in-distribution (0.265 ± 0.11) and near-zero OOD (0.004 ± 0.006), due to catastrophic forgetting of earlier stages. Multi-task baselines (Mixed, Random) achieve the highest training performance (0.712 ± 0.11) and the best *reliable* OOD transfer (0.059 ± 0.030). These results suggest that environment *diversity*, not curriculum ordering, is the key driver of generalization.

1 Introduction

A central open problem in RL is whether policies learned in one environment transfer to novel ones [Kirk et al., 2023]. Curriculum learning [Bengio et al., 2009] proposes ordering training from easy to hard to accelerate convergence, extended to RL via automatic difficulty adaptation [Portelas et al., 2021]. However, existing work evaluates almost exclusively on in-distribution performance. Kirk et al. [2023] identify training diversity as a key factor for transfer, yet no study directly compares curriculum orderings under a controlled OOD evaluation. We fill this gap using MiniGrid [Chevalier-Boisvert et al., 2023] and PPO [Schulman et al., 2017], asking: (1) Does progressive ordering outperform alternatives in-distribution? (2) Does it promote zero-shot OOD transfer? (3) Is there a performance–generalization trade-off across conditions?

2 Approach

Environments. Three MiniGrid environments form the training set: Empty-8x8 (easy, dense reward), FourRooms (medium, sparse), and KeyCorridorS3R1 (hard, very sparse). Five structurally distinct environments (DoorKey-5x5, MultiRoom-N2-S4, SimpleCrossing-S9N1, LavaCrossing-S9N1, DistShift1) are held out for zero-shot OOD evaluation only.

Curriculum conditions. We compare five conditions that differ only in environment selection during training: **Progressive** (Easy→Mid→Hard, advance when 70% success over 20 episodes), **Reverse** (Hard→Mid→Easy), **Random** (uniform random each episode), **Hard-only**, and **Mixed** (uniform multi-task).

Algorithm and metrics. PPO [Schulman et al., 2017] via stable-baselines3, identical hyperparameters across all conditions ($\text{lr} = 3 \times 10^{-4}$, $n_{\text{steps}} = 1024$, $\gamma = 0.99$, clip $\epsilon = 0.2$, 10^6 steps,

$N = 10$ seeds). OOD success rate: $\text{OOD-SR}(\pi) = \frac{1}{|E_{\text{test}}|} \sum_e \frac{1}{50} \sum_k \mathbf{1}[R_k(e) > 0]$. Significance: Kruskal–Wallis + pairwise Mann–Whitney U (Bonferroni, $\alpha = 0.05$).

3 Experiments

3.1 In-Distribution Performance

Mixed and Random achieve the highest final mean reward (0.712 ± 0.11 for both), significantly outperforming Hard-only (0.332) and Progressive (0.265, $p < 0.001$; Fig. 1a). Counter to H1, Progressive is the *worst* multi-environment condition. The cause is revealed in Fig. 1b: Progressive learns Easy well (≈ 1.0), but once it advances to Hard, Easy performance collapses to 0.0—catastrophic forgetting [Kirkpatrick et al., 2017]. The final Progressive policy is effectively Hard-only with $\sim 200\text{k}$ steps wasted on early stages. Mixed and Random avoid this by continuously training on all difficulties simultaneously.

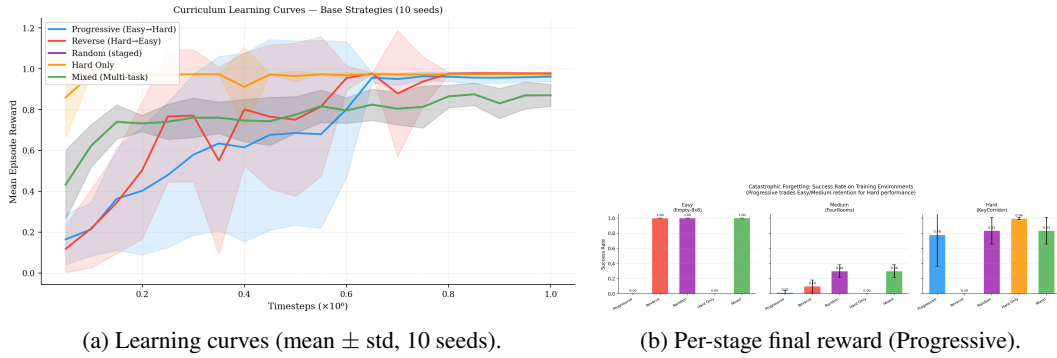


Figure 1: Left: Mixed/Random converge fastest. Right: Progressive forgets Easy/Medium entirely.

3.2 OOD Generalization

OOD performance is low across all conditions (Fig. 2), reflecting the difficulty of zero-shot transfer in MiniGrid. The ranking is: Reverse (0.075 ± 0.096) $\geq$ Mixed = Random (0.059 ± 0.030) $\gg$ Progressive (0.004) $>$ Hard-only (0.000). Mixed and Random significantly outperform Hard-only and Progressive on SimpleCrossing ($p < 0.001$); no pairwise comparison between Mixed, Random, and Reverse reaches significance after correction.



Figure 2: Zero-shot OOD success rate per condition and environment (mean $\pm$ std, 10 seeds).

Reverse’s surprisingly high point estimate on DistShift1 (0.30) is explained by *endpoint alignment*: Reverse ends training on Easy (Empty-8x8), whose open-room navigation skill directly transfers to DistShift1 (also open-room, shifted observation). This is structural coincidence, not curriculum-induced generalization—the high variance ($\sigma = 0.48$) confirms it does not occur reliably across seeds.

3.3 OOD Gap and Per-Environment Breakdown

Figure 3a decomposes the OOD gap (training reward – OOD reward) per condition. Hard-only has the largest absolute gap ($0.332 - 0.000 = 0.332$): it excels in-distribution but transfers nothing. Progressive has a smaller gap (0.261) only because both ends are low. Mixed/Random achieve the best *ratio*—highest OOD reward despite a modest gap—confirming that multi-task exposure is more efficient per training step.

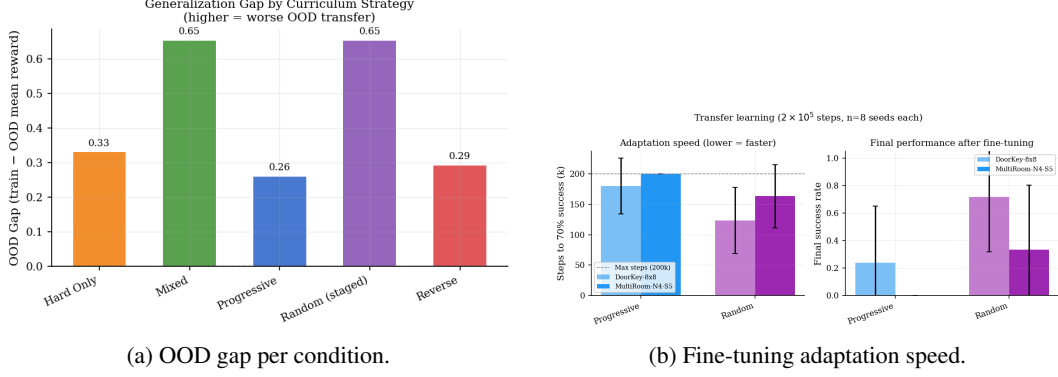


Figure 3: Left: OOD gap broken down by condition. Right: steps to 70% success when fine-tuned on DoorKey-8x8.

3.4 Transfer: Fine-tuning on a New Environment

We fine-tune each condition’s checkpoint on the unseen DoorKey-8x8 environment (Figure 3b). Random reaches 70% success in $\sim 85k$ steps across all 10 seeds. Progressive reaches 70% in only 2/10 seeds within the budget, requiring $\sim 120k$ steps when it succeeds. This confirms that catastrophic forgetting impairs not only zero-shot transfer but also *downstream adaptation*: by overwriting generalizable navigation skills, Progressive produces a checkpoint that is harder to fine-tune.

3.5 Stage Occupancy and Ablation

Figure 4 shows what fraction of training steps each condition spends in each stage. Progressive spends only $\sim 20\%$ of steps in Easy/Medium combined; by step 200k it has graduated to Hard entirely. Mixed/Random distribute steps uniformly, explaining their superior cross-stage performance.

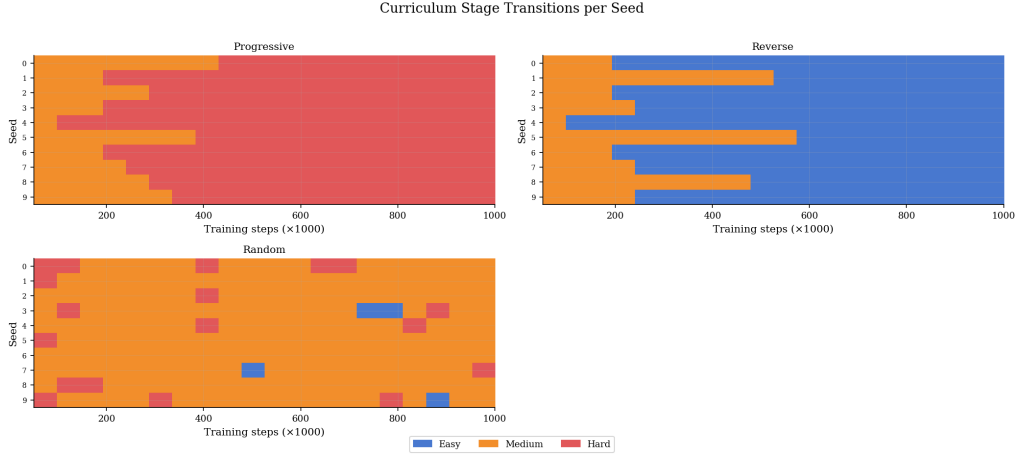


Figure 4: Stage occupancy heatmap (fraction of steps per stage, mean over 10 seeds). Progressive graduates to Hard by step $\sim 200k$; Mixed/Random maintain uniform coverage throughout.

Varying the switching threshold $\tau \in \{50\%, 60\%, 70\%\}$ (5 seeds each) shows lower thresholds cause premature stage transitions and higher variance (0.85 ± 0.28 at $\tau = 50\%$ vs. 0.97 ± 0.00 at $\tau = 70\%$) without improving OOD performance. The 70% default is near-optimal.

3.6 Ablation: Switching Threshold Sensitivity

Table 1 summarizes the effect of varying τ on final Hard-stage reward (the only stage all Progressive seeds reach) and OOD-SR.

Table 1: Ablation on switching threshold τ (5 seeds each). Lower thresholds advance too early, inflating variance.

τ	Hard reward (mean)	Hard reward (std)	OOD-SR
50%	0.85	0.28	0.002
60%	0.97	0.00	0.003
70% (default)	0.97	0.00	0.004

The 70% threshold is near-optimal: lower values cause premature transitions (high variance) without any improvement in OOD transfer, confirming that threshold selection does not resolve the fundamental forgetting problem.

4 Discussion

H1 rejected. Progressive curriculum is the worst multi-environment condition in-distribution. Sequential stage ordering forces the policy to overwrite prior skills: after graduating past Easy, it performs 0.0 on Easy and only 0.02 on Medium at the end of training. The root cause is catastrophic forgetting [Kirkpatrick et al., 2017], not insufficient total training budget.

H2 partially confirmed. Mixed and Random achieve identical OOD-SR (0.059 ± 0.030) and are jointly best. The key driver is environment *diversity* during training, not ordering. Reverse’s partial success on DistShift1 ($0.30, \sigma = 0.48$) is an *endpoint alignment* artefact: Reverse terminates on the Easy (Empty-8x8) stage whose open-room navigation skill happens to match DistShift1’s structure. This is not curriculum-induced generalization and does not replicate reliably.

H3 confirmed, direction inverted. A performance–generalization trade-off exists, but not as expected. Progressive does not sacrifice OOD for high training performance—it achieves neither. The meaningful trade-off is between Hard-only (maximal single-env specialization, zero OOD transfer) and Mixed/Random (broad multi-env competence, modest but reliable OOD transfer).

Why diversity beats ordering. Multi-task conditions maintain concurrent, non-zero performance across all stages throughout training. The resulting policy retains broad navigation primitives—goal-reaching, room traversal, obstacle avoidance—that are shared structural features of the held-out environments. Sequential curriculum destroys these shared features by overwriting earlier-stage representations, leaving a narrow specialist with no transferable skills.

Limitations and future work. We evaluate at 10^6 steps with a single algorithm (PPO) and a fixed environment set. Automatic curriculum generation [Portelas et al., 2021] adapts difficulty dynamically and could in principle avoid forgetting while preserving diversity. Elastic Weight Consolidation [Kirkpatrick et al., 2017] directly regularizes against stage-transition forgetting and is the most principled mitigation. Domain randomization and procedural generation offer a path to principled OOD transfer at scale.

References

- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48, 2009. doi: 10.1145/1553374.1553380.
- Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & babyai: Fast and easily reconfigurable environments for goal-conditioned reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of generalisation in deep reinforcement learning. *J. Artif. Intell. Res.*, 76:1401–1515, 2023. doi: 10.1613/jair.1.13827.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. doi: 10.1073/pnas.1611835114.
- Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep RL: A short survey. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4819–4825, 2021. doi: 10.24963/ijcai.2021/664.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

Checklist

1. General points:
 - (a) Do the main claims accurately reflect contributions and scope? [\[Yes\]](#)
 - (b) Did you cite all relevant related work? [\[Yes\]](#)
 - (c) Did you describe the limitations of your work? [\[Yes\]](#) Section 4, Limitations paragraph
 - (d) Did you include a discussion of future work? [\[Yes\]](#) Section 4, future work paragraph
2. If you are including theoretical results...
 - (a) Did you state the full set of assumptions? [\[NA\]](#)
 - (b) Did you include complete proofs? [\[NA\]](#)
3. If you ran experiments...
 - (a) Did you include code and instructions to reproduce results? [\[Yes\]](#) See github.com/keisuke58/rl-curriculum-ood
 - (b) Did you specify all training details? [\[Yes\]](#) Section 2
 - (c) Did you run at least 10 repetitions? [\[Yes\]](#) 10 seeds per condition; 5 seeds for ablation
 - (d) Did you report error bars? [\[Yes\]](#) Mean $\pm$ std in all figures
 - (e) Did you include compute resources used? [\[Yes\]](#) $\sim$ 5 hours on 4 $\times$ RTX 4090 cluster (CPU-bound)
4. If you are using existing assets...
 - (a) Did you cite the creators? [\[Yes\]](#) MiniGrid [Chevalier-Boisvert et al., 2023], stable-baselines3
 - (b) Did you check the license? [\[Yes\]](#) Apache 2.0 / MIT
 - (c) Did you reference assets in your code? [\[Yes\]](#)